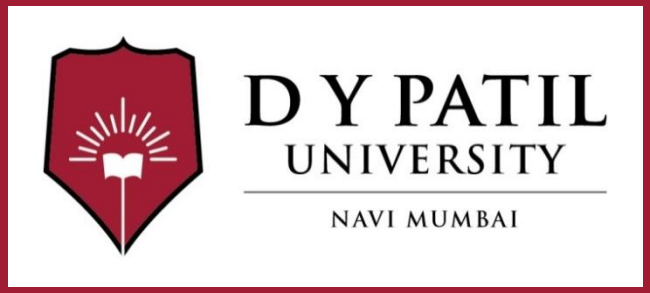
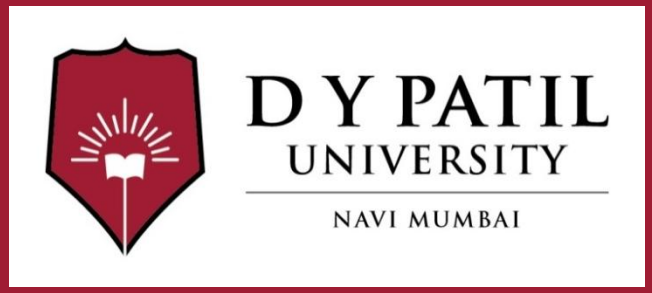


Subject Name : ML

Unit No: 3

Unit Name: Tree-Based Models and Ensemble Learning

Faculty Name : Dr. Ritu Jain



Unit No.: 3.2

Faculty Name :

Example of AdaBoost

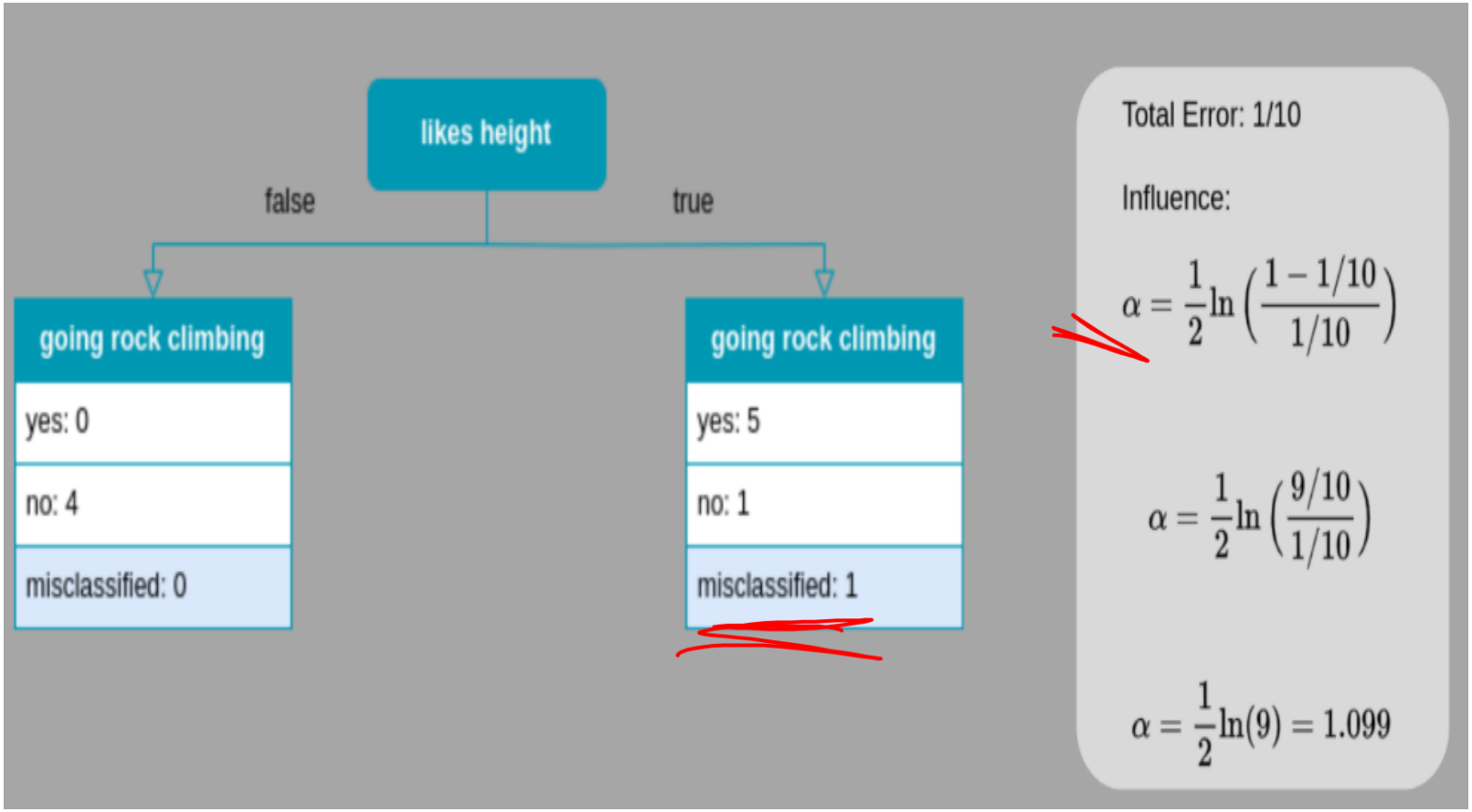
- The dataset used in this example contains only 10 samples, to make the calculations by hand more feasible. It describes the problem of whether a person should go rock climbing or not, depending on their age, and whether the person likes height and goats.

age	likes height	likes goats	go rock climbing
23	0	0	0
31	1	1	1
35	1	0	1
35	0	0	0
42	0	0	0
43	1	1	1
45	0	1	0
46	1	1	1
46	1	0	0
51	1	1	1

Example of AdaBoost

- Our ensemble model will consist of three weak learners. It will be decision *stump*.
- The first step in building an AdaBoost model is assigning weights to the individual data points. In the beginning, for the initial model, all data points get the same weight assigned, which is 1/N , with N the dataset size.

AdaBoost-First Decision Stump



Example of AdaBoost

With the *influence* α calculated, we can determine the new weights for the next iteration.

$$w_{new} = w_{old} \cdot e^{\pm\alpha}.$$

The sign in this equation depends on whether a sample was correctly classified or not. For correctly classified samples, we get

$$w_{new} = 0.1 \cdot e^{-1.099} = 0.0333,$$

and for wrongly classified samples

$$w_{new} = 0.1 \cdot e^{1.099} = 0.3,$$

accordingly. These weights still need to be normalized, so that their sum equals 1. This is done, by dividing by their sum. The next plot shows the dataset with the new weights.

Example of AdaBoost

- **The dataset with Updated weights based on result of first stump and the influence .**

age	likes height	likes goats	go rock climbing	weight	normalized weight
23	0	0	0	0.033	0.056
31	1	1	1	0.033	0.056
35	1	0	1	0.033	0.056
35	0	0	0	0.033	0.056
42	0	0	0	0.033	0.056
43	1	1	1	0.033	0.056
45	0	1	0	0.033	0.056
46	1	1	1	0.033	0.056
46	1	0	0	0.3	0.5
51	1	1	1	0.033	0.056

Example of AdaBoost

The weights are used to create bins. Let's assume we have the weights $w_1, w_2, \ldots, w_N$, the bin for the first sample is $[0, w_1]$, for the second sample, $[w_1, w_1 + w_2]$, etc. In our example, the bin of the first sample is $[0, 0.056]$, for the second $[0.056, 0.112]$, etc.. The following plot shows all samples with their bins.

The dataset with bins based on the weights.

age	likes height	likes goats	go rock climbing	bins
23	0	0	0	[0,0.056]
31	1	1	1	[0.056, 0.111]
35	1	0	1	[0.111, 0.178]
35	0	0	0	[0.178, 0.222]
42	0	0	0	[0.222, 0.278]
43	1	1	1	[0.278, 0.333]
45	0	1	0	[0.333, 0.389]
46	1	1	1	[0.389, 0.444]
46	1	0	0	[0.444, 0.944]
51	1	1	1	[0.944, 1]

Example of AdaBoost

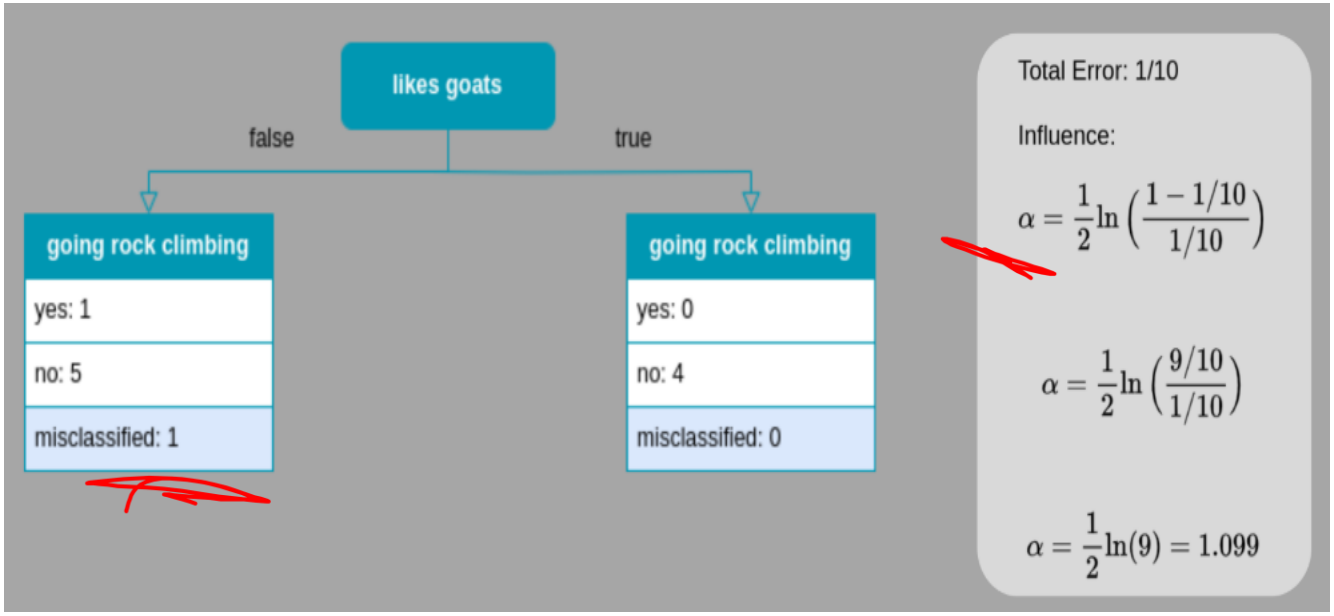
Modified dataset to build second stump

Now, some randomness comes into play. Random numbers between 0 and 1 are drawn, then we check in which bin the random number falls, and the according data sample is selected for the new modified dataset. We draw as many numbers as the length of our dataset is, that is in this example we draw 10 numbers. Due to the higher weight of the misclassified example, this example has a larger bin, and the probability of drawing it is higher. Let's assume we draw the numbers $[0.1, 0.15, 0.06, 0.5, 0.65, 0.05, 0.8, 0.7, 0.95, 0.97]$, which leads to the selection of the samples $[1, 2, 1, 8, 8, 0, 8, 8, 9, 9]$. The modified dataset then has the following form.

age	likes height	likes goats	go rock climbing
31	1	1	1
23	1	0	1
31	1	1	1
46	1	0	0
46	1	0	0
23	0	0	0
46	1	0	0
46	1	0	0
51	1	1	1
51	1	1	1

Example of AdaBoost

- We now use this modified dataset to create the second stump. Additionally, the influence of this stump is calculated.
- ***The second stump, i.e. the second weak learner for our AdaBoost algorithm.***



Example of AdaBoost

As in the first stump, one sample is misclassified, so we get the same value for alpha as for the first stump. Accordingly, the weights are updated using the above formula.

$$w_{new} = w_{old} \cdot e^{\pm\alpha}.$$

For the first sample, this results in

$$w_{new} = 0.056 \cdot e^{-1.099} = 0.0185.$$

The second sample was the only one misclassified, so the sign in the exponent needs to be changed

$$w_{new} = 0.056 \cdot e^{1.099} = 0.167.$$

Example of AdaBoost

- The following plot shows all samples together with their old weights, new weights, and normalized weights.

age	likes height	likes goats	go rock climbing	old weight	new weight	normalized weight
31	1	1	1	0.056	0.0185	0.02
35	1	0	1	0.056	0.167	0.18
31	1	1	1	0.056	0.0185	0.02
46	1	0	0	0.5	0.167	0.18
46	1	0	0	0.5	0.167	0.18
23	0	0	0	0.056	0.0185	0.02
46	1	0	0	0.5	0.167	0.18
46	1	0	0	0.5	0.167	0.18
51	1	1	1	0.056	0.0185	0.02
51	1	1	1	0.056	0.0185	0.02

Example of AdaBoost

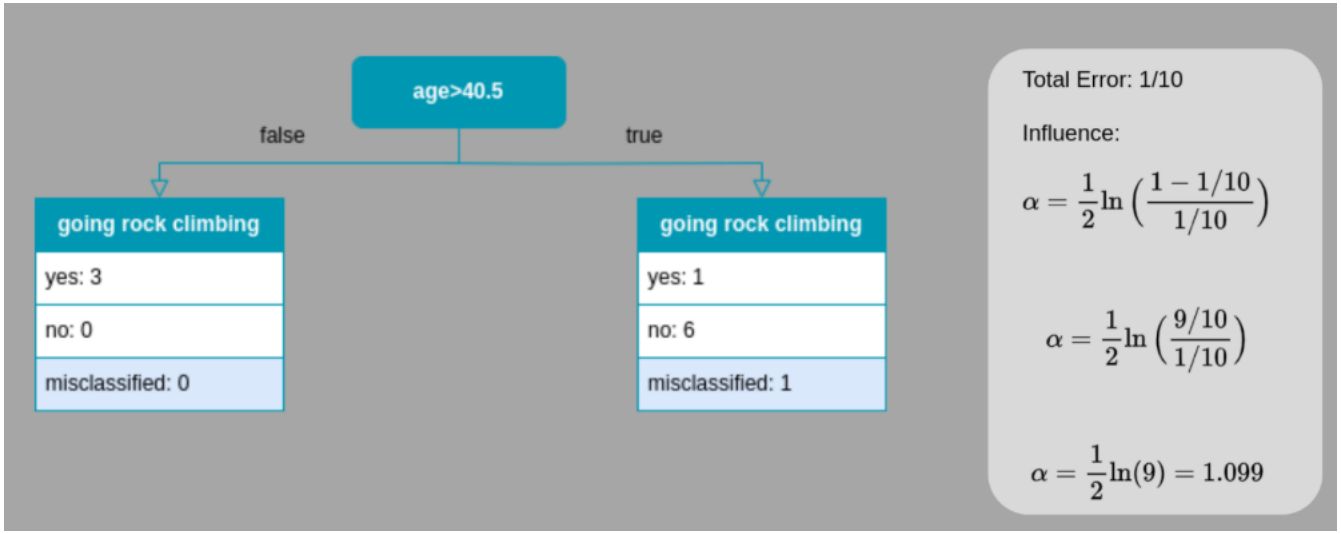
- As in the first iteration, we convert the weights into bins.

We repeat the bootstrapping and draw 10 random numbers between 0 and 1. Let's assume we draw the numbers [0.3, 0.35, 0.1, 0.4, 0.97, 0.8, 0.9, 0.05, 0.25, 0.05], which refer to the samples [4, 4, 1, 4, 9, 7, 7, 1, 3, 1], then we get the following modified dataset.

age	likes height	likes goats	go rock climbing
46	1	0	0
46	1	0	0
35	1	0	1
46	1	0	0
51	1	1	1
46	1	0	0
46	1	0	0
35	1	0	0
46	1	0	0
35	1	0	1

Example of AdaBoost

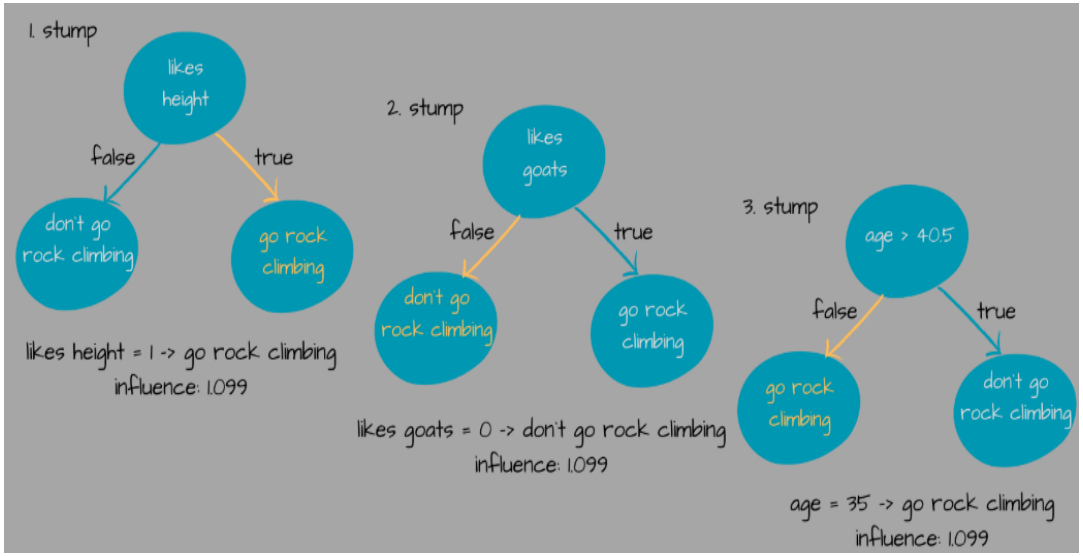
- *Modified dataset based on the weights.*
- We can now fit the third and last stump of our model to this modified dataset and calculate its influence. The result is shown in the next plot.
- *The third stump, i.e. the last weak learner for our AdaBoost algorithm.*



Example of AdaBoost

- Again, one sample is misclassified, which leads to the same influence as previously. Note, that this is due to the very simplified dataset considered, in a more realistic example the influences of the different models would differ. We now use the individual trees and their calculated values to determine the final prediction. Let's consider one of the samples in the dataset.
- We now make predictions for each of the three stumps for this sample.

Feature	Value
age	35
likes height	1
likes goats	0



Example of AdaBoost

The final prediction is achieved by adding up the influences of each tree for the predicted classes. In this example the first and the third stump predict "go rock climbing" and the second stump predicts "don't go rock climbing". The first and the third stump have an influence of $1.099 + 1.099 = 2.198$, and the second stump has an influence of 1.099. That means the influence for the prediction "go rock climbing" is higher and this is thus our final prediction.

The predictions and their influences to determine the final prediction.

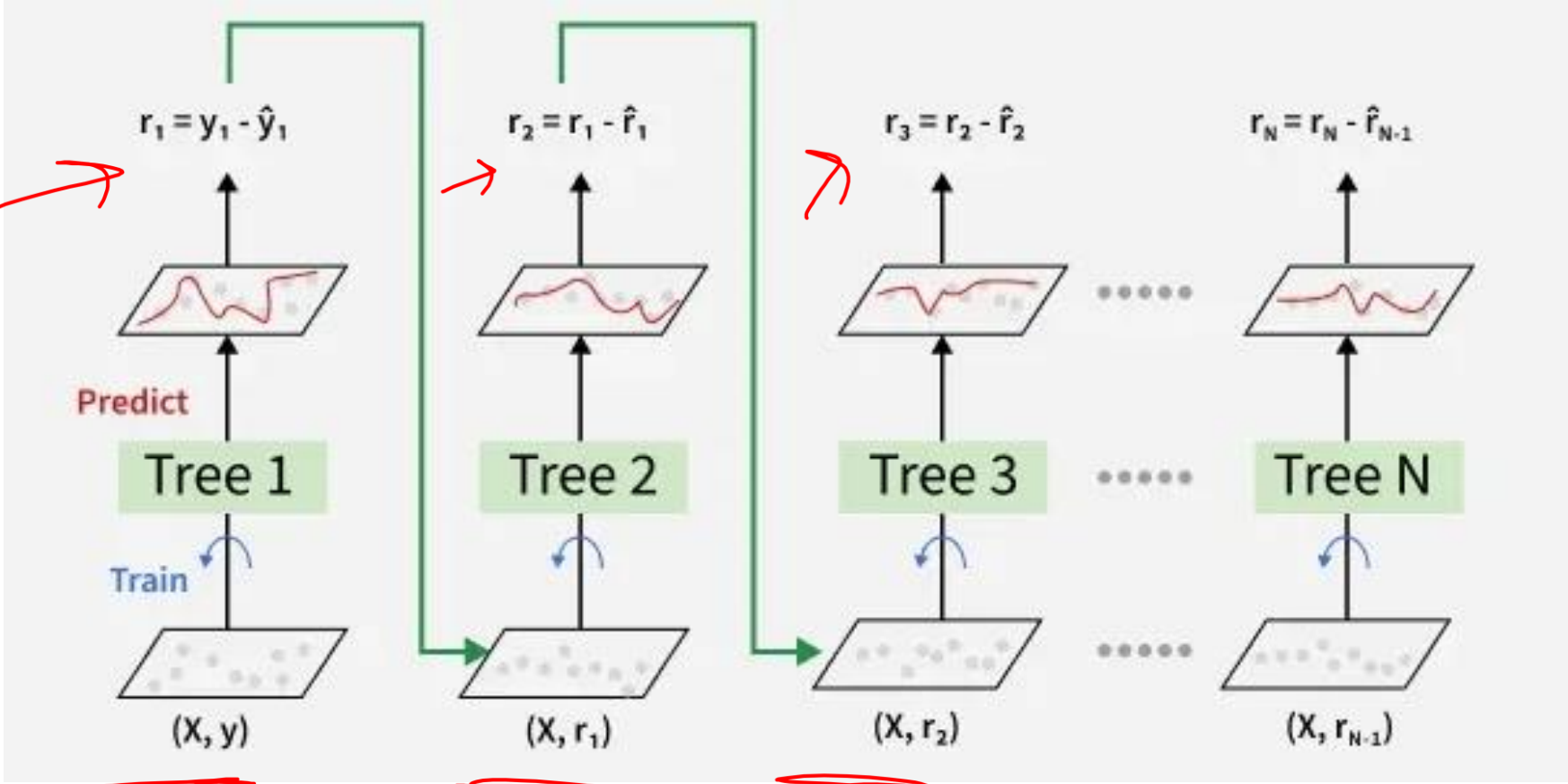
Sample: age = 35
likes height = 1
likes goats = 0

stump \ predicted class	go rock climbing	don't go rock climbing
influence of stump 1	1.099	
influence of stump 2		1.099
influence of stump 3	1.099	
sum of influences	2.198	1.099

Gradient Boosting

- Gradient Boosting is a **boosting** algorithm and here each new model is trained to minimize the loss function such as mean squared error or cross-entropy of the previous model using gradient descent.
- In each iteration the algorithm computes the gradient of the loss function with respect to predictions and then trains a new weak model to predict this gradient.
- Predictions of the new model are then added to the ensemble (all models prediction) and the process is repeated until a stopping criterion is met.

Working of Gradient Boosting



Working of Gradient Boosting

1. Sequential Learning Process: The ensemble consists of multiple trees each trained to correct the errors of the previous one. In the first iteration **Tree 1** is trained on the original data x and the true labels y. It makes predictions which are used to compute the errors.

2. Residuals Calculation: In the second iteration **Tree 2** is trained using the feature matrix x and the errors from Tree 1 as labels. This means Tree 2 is trained to predict the errors of Tree 1. This process continues for all the trees in the ensemble. Each subsequent tree is trained to predict the errors of the previous tree.

3. Shrinkage: After each tree is trained its predictions are **shrunk** by multiplying them with the learning rate η which ranges from 0 to 1. This prevents overfitting by ensuring each tree has a smaller impact on the final model.

Once all trees are trained predictions are made by summing the contributions of all the trees. The final prediction is given by the formula:

$$y_{pred} = y_1 + \eta \cdot r_1 + \eta \cdot r_2 + \cdots + \eta \cdot r_N$$

Where $r_1, r_2, \ldots, r_N$ are the errors predicted by each tree.

XGBoost

- XGBoost (eXtreme Gradient Boosting) is an optimized, high-performance implementation of Gradient Boosting Machines (GBM), designed for speed and accuracy.
- While both build trees sequentially, XGBoost improves on traditional GBM by adding regularization (L1/L2) to prevent overfitting, using parallel processing to reduce training time, and offering native handling of missing values.

Key Features and Optimizations of XGBoost

- What makes XGBoost "extreme" are several system and algorithmic optimizations that enhance performance and efficiency compared to traditional gradient boosting:
- **Regularization:** XGBoost includes built-in L1 (Lasso) and L2 (Ridge) regularization terms in its objective function to penalize complex models and prevent overfitting, a common issue in tree-based methods.
- **Parallel Processing:** It uses all CPU cores during training by storing data in compressed, cache-aware internal memory units called "blocks," which allows for parallel construction of the trees one level at a time (level-wise growth).
- **Handling Missing Values:** The algorithm has a sparsity-aware approach that automatically learns the best direction for missing values to take during a split, without requiring explicit imputation.

Key Features and Optimizations of XGBoost

- **Tree Pruning**: XGBoost builds trees to a maximum depth and then prunes branches backward if they don't meet a minimum loss reduction (controlled by the gamma parameter), which simplifies the trees and reduces overfitting.
- **Cache Optimization:** It has been designed to make optimal use of hardware, using CPU cache memory to store gradient statistics and reduce memory access time, leading to faster training.
- **Flexibility**: It supports custom loss functions and can be used for a variety of tasks including classification, regression, and ranking problems.
- **Sparsity-Aware Split Finding**: For large datasets with many potential split points, XGBoost uses an approximate greedy algorithm with weighted quantile sketches to quickly estimate the best splits, reducing computational overhead while maintaining accuracy

How XGBoost Works?

- It builds decision trees sequentially with each tree attempting to correct the mistakes made by the previous one. The process can be broken down as follows:
- **Start with a base learner**: The first model decision tree is trained on the data. In regression tasks this base model simply predicts the average of the target variable.
- **Calculate the errors**: After training the first tree the errors between the predicted and actual values are calculated.
- **Train the next tree**: The next tree is trained on the errors of the previous tree. This step attempts to correct the errors made by the first tree.
- **Repeat the process**: This process continues with each new tree trying to correct the errors of the previous trees until a stopping criterion is met.
- **Combine the predictions**: The final prediction is the sum of the predictions from all the trees.

When to Use XGBoost

- **Structured/Tabular Data:** Use it for data stored in CSVs or SQL databases. It outperforms deep learning on these datasets.
- **Large Datasets:** It is highly scalable, supporting parallel processing and GPU acceleration to handle millions of records efficiently.
- **Missing Data & Sparsity:** Use it when your dataset has many missing values or sparse features; XGBoost has built-in sparsity-aware algorithms that handle them automatically.
- **Complex Relationships:** It excels at capturing non-linear patterns and feature interactions that simpler models like logistic regression might miss.
- **Need for Regularization:** Use it if your model is prone to overfitting; it includes L1 (Lasso) and L2 (Ridge) regularization to keep the model simple and robust.
- **Ranking Tasks:** It is effective for "Learning to Rank" problems, such as search engine results or recommendation systems.

When to Avoid XGBoost

- **Unstructured Data:** Do not use it for images, audio, or natural language processing (NLP). Deep learning and neural networks are significantly better for these domains.
- **Very Small Datasets:** For datasets with fewer than 100 samples, it may overfit easily.
- **High Interpretability Needs:** While it provides feature importance, it is more of a "black box" than linear models or simple decision trees.
- **High Feature-to-Sample Ratio:** It is less effective when the number of features is significantly larger than the number of training samples

Prepare the following topics

- Difference between
  - Decision Tree and Random Forest
  - AdaBoost and XGBoost
  - Random Forest and AdaBoost
  - and other combination of the above ML models.

Hyperparameter Tuning

- Hyperparameter tuning is the process of selecting the optimal values for a machine learning model's hyperparameters. These are typically set before the actual training process begins and control aspects of the learning process itself.
- Effective tuning helps the model learn better patterns, avoid overfitting or underfitting and achieve higher accuracy on unseen data.

Techniques for Hyperparameter Tuning

- Models can have many hyperparameters and finding the best combination of parameters can be treated as a search problem.
- One of the strategy for Hyperparameter tuning is GridSearchCV

GridSearchCV

- **Grid search** is an exhaustive, brute-force hyperparameter tuning technique that systematically evaluates every possible combination of values provided in a predefined grid.
- It is widely used to find the optimal configuration for a machine learning model by training and testing each variation to identify the one that yields the best performance score, typically measured through **cross-validation**.

GridSearchCV

- It works using below steps:
  - Create a grid of potential values for each hyperparameter.
  - Train the model for every combination in the grid.
  - Evaluate each model using cross-validation.
  - Select the combination that gives the highest score.

GridSearchCV

- For example if we want to tune two hyperparameters C and Alpha for a machine learning model with the following sets of values:
  C = [0.1, 0.2, 0.3, 0.4, 0.5]
  Alpha = [0.01, 0.1, 0.5, 1.0]

C \ Alpha	0.1	0.2	0.3	0.4
0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675

- The grid search technique will construct multiple versions of the model with all possible combinations of C and Alpha, resulting in a total of 5 * 4 = 20 different models. The best-performing combination is then chosen.

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI